Guide des Annotations Spring

🏗️ Configuration et Démarrage

`@SpringBootApplication`

- **Usage**: Classe principale de l'application
- **Combine**: `@Configuration`, `@EnableAutoConfiguration`, `@ComponentScan`

```java
@SpringBootApplication
public class Application {
    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }
}
```

`@Configuration`

- **Usage**: Classe de configuration
- **Rôle**: Définit des beans via des méthodes `@Bean`

```java
@Configuration
public class AppConfig {
    @Bean
    public DataSource dataSource() {
        return new HikariDataSource();
    }
}
```

`@Bean`

- **Usage**: Méthode dans une classe `@Configuration`
- **Rôle**: Crée et configure un bean

```java
@Bean
public RestTemplate restTemplate() {
    return new RestTemplate();
}
```

🔍 Détection des Composants

`@ComponentScan`

- **Usage**: Classe de configuration
- **Rôle**: Définit les packages à scanner pour les composants

```java
@ComponentScan(basePackages = {"service", "repository"})
```

`@Component`

- **Usage**: Classe générique
- **Rôle**: Bean générique détecté par `@ComponentScan`

```java
@Component
public class EmailService {
    // ...
}
```

`@Service`

- **Usage**: Classe de service (logique métier)
- **Rôle**: Spécialisation de `@Component`

```java
@Service
public class UserService {
    // ...
}
```

`@Repository`

- **Usage**: Classe d'accès aux données
- **Rôle**: Spécialisation de `@Component` + gestion des exceptions

```java
@Repository
public interface UserRepository extends JpaRepository<User, Long> {
    // ...
}
```

@Controller

- **Usage**: Classe de contrôleur web
- **Rôle**: Spécialisation de `@Component` pour les contrôleurs

```java
@Controller
public class UserController {
    // ...
}
```

@RestController

- **Usage**: Classe de contrôleur REST
- **Rôle**: Combine `@Controller` + `@ResponseBody`

```java
@RestController
public class UserRestController {
    // ...
}
```

💉 Injection de Dépendances

@Autowired

- **Usage**: Champ, constructeur, méthode
- **Rôle**: Injection automatique par type

```java
@Autowired
private UserService userService;
```

@Qualifier

- **Usage**: Avec `@Autowired`
- **Rôle**: Précise quel bean injecter quand plusieurs candidats

```java
@Autowired
@Qualifier("primaryUserService")
private UserService userService;
```

`@Primary`

- **Usage**: Bean ou méthode `@Bean`
- **Rôle**: Bean prioritaire en cas de conflit

```java
@Service
@Primary
public class PrimaryUserService implements UserService {
    // ...
}
```

`@Value`

- **Usage**: Champ ou paramètre
- **Rôle**: Injection de valeurs depuis properties

```java
@Value("${app.name}")
private String appName;
```

🌐 Web et REST

`@RequestMapping`

- **Usage**: Classe ou méthode de contrôleur
- **Rôle**: Mapping des requêtes HTTP

```java
@RequestMapping("/users")
public class UserController {
    @RequestMapping(method = RequestMethod.GET)
    public List<User> getUsers() { }
}
```

`@GetMapping`, `@PostMapping`, `@PutMapping`, `@DeleteMapping`

- **Usage**: Méthode de contrôleur
- **Rôle**: Raccourcis pour les verbes HTTP

```java
@GetMapping("/users/{id}")
public User getUser(@PathVariable Long id) { }

@PostMapping("/users")
public User createUser(@RequestBody User user) { }
```

`@PathVariable`

- **Usage**: Paramètre de méthode
- **Rôle**: Extrait une variable de l'URL

```java
@GetMapping("/users/{id}")
public User getUser(@PathVariable Long id) { }
```

`@RequestParam`

- **Usage**: Paramètre de méthode
- **Rôle**: Extrait un paramètre de query string

```java
@GetMapping("/users")
public List<User> getUsers(@RequestParam(defaultValue = "0") int page) { }
```

`@RequestBody`

- **Usage**: Paramètre de méthode
- **Rôle**: Désérialise le corps de la requête

```java
@PostMapping("/users")
public User createUser(@RequestBody User user) { }
```

`@ResponseBody`

- **Usage**: Méthode de contrôleur
- **Rôle**: Sérialise la réponse (inclus dans `@RestController`)

```java
@Controller
public class UserController {
    @GetMapping("/users")
    @ResponseBody
    public List<User> getUsers() { }
}
```

🗄️ Données (JPA/MongoDB)

@EnableJpaRepositories

- **Usage**: Classe de configuration
- **Rôle**: Active les repositories JPA

```java
@EnableJpaRepositories(basePackages = "repository")
```

@EnableMongoRepositories

- **Usage**: Classe de configuration
- **Rôle**: Active les repositories MongoDB

```java
@EnableMongoRepositories(basePackages = "repository")
```

@Entity

- **Usage**: Classe JPA
- **Rôle**: Marque une entité JPA

```java
@Entity
public class User {
    @Id
    private Long id;
}
```

@Document

- **Usage**: Classe MongoDB
- **Rôle**: Marque un document MongoDB

```java
@Document(collection = "users")
public class User {
    @Id
    private String id;
}
```

📊 Validation

`@Valid`

- **Usage**: Paramètre de méthode
- **Rôle**: Active la validation

```java
@PostMapping("/users")
public User createUser(@Valid @RequestBody User user) { }
```

`@NotNull`, `@NotBlank`, `@Size`, etc.

- **Usage**: Champs d'entité
- **Rôle**: Contraintes de validation

```java
public class User {
    @NotBlank
    @Size(min = 2, max = 50)
    private String name;

    @Email
    private String email;
}
```

🎯 Aspect-Oriented Programming (AOP)

`@EnableAspectJAutoProxy`

- **Usage**: Classe de configuration
- **Rôle**: Active le support AOP

java

```java
@Configuration
@EnableAspectJAutoProxy
public class AopConfig { }
```

@Aspect

- **Usage**: Classe d'aspect
- **Rôle**: Définit un aspect

java

```java
@Aspect
@Component
public class LoggingAspect {
    @Before("execution(* service.*.*(..))")
    public void logBefore() { }
}
```

📝 Messaging (JMS, RabbitMQ, etc.)

@EnableJms

- **Usage**: Classe de configuration
- **Rôle**: Active le support JMS

java

```java
@EnableJms
@Configuration
public class JmsConfig { }
```

@JmsListener

- **Usage**: Méthode
- **Rôle**: Écoute les messages JMS

java

```java
@JmsListener(destination = "my-queue")
public void handleMessage(String message) { }
```

🔄 Transactions

@Transactional

- **Usage**: Classe ou méthode
- **Rôle**: Gestion des transactions

```java
@Service
@Transactional
public class UserService {
    public void createUser(User user) { }
}
```

@EnableTransactionManagement

- **Usage**: Classe de configuration
- **Rôle**: Active la gestion des transactions

```java
@Configuration
@EnableTransactionManagement
public class TransactionConfig { }
```

🧪 Tests

@SpringBootTest

- **Usage**: Classe de test
- **Rôle**: Charge le contexte Spring complet

```java
@SpringBootTest
class UserServiceTest {
    @Autowired
    private UserService userService;
}
```

@WebMvcTest

- **Usage**: Classe de test
- **Rôle**: Test des contrôleurs web uniquement

```java
@WebMvcTest(UserController.class)
class UserControllerTest {
    @MockBean
    private UserService userService;
}
```

@DataJpaTest

- **Usage**: Classe de test
- **Rôle**: Test des repositories JPA

```java
@DataJpaTest
class UserRepositoryTest {
    @Autowired
    private UserRepository userRepository;
}
```

@MockBean

- **Usage**: Champ de test
- **Rôle**: Crée un mock dans le contexte Spring

```java
@MockBean
private UserService userService;
```

🔧 Conditionnelles

@ConditionalOnProperty

- **Usage**: Bean ou configuration
- **Rôle**: Conditionne la création sur une propriété

```java
@Bean
@ConditionalOnProperty(name = "feature.enabled", havingValue = "true")
public FeatureService featureService() { }
```

@ConditionalOnClass

- **Usage**: Bean ou configuration
- **Rôle**: Conditionne la création sur la présence d'une classe

```java
@Bean
@ConditionalOnClass(DataSource.class)
public DatabaseService databaseService() { }
```

🎨 Profils

`@Profile`

- **Usage**: Bean ou configuration
- **Rôle**: Active selon le profil Spring

```java
@Service
@Profile("production")
public class ProductionEmailService implements EmailService { }
```

`@ActiveProfiles`

- **Usage**: Classe de test
- **Rôle**: Active des profils pour les tests

```java
@SpringBootTest
@ActiveProfiles("test")
class UserServiceTest { }
```

📋 Conseils d'utilisation

1. **Privilégier les annotations spécialisées** : `@Service`, `@Repository`, `@Controller` plutôt que `@Component`
2. **Utiliser `@Primary` et `@Qualifier`** pour gérer les conflits de beans
3. **Préférer l'injection par constructeur** à `@Autowired` sur les champs
4. **Utiliser `@Value` avec des valeurs par défaut** : `@Value("${prop:default}")`
5. **Grouper les configurations** dans des classes `@Configuration` séparées